

LCR 021. 删除链表的倒数第 N 个结点

已解答

中等 相关标签 相关企业 Aa

给定一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

示例 1:

输入: head = [1,2,3,4,5], n = 2
输出: [1,2,3,5]

示例 2:

输入: head = [1], n = 1
输出: []

```

1  /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* removeNthFromEnd(ListNode* head, int n) {
14         // 定义一个哑结点
15         // ListNode(int x, ListNode *next) : val(x), next(next) {}
16         ListNode* dummy = new ListNode(0, head);
17         // 定义一个栈
18         stack<ListNode*> stk;
19         ListNode* cur = dummy;
20         // 循环入栈
21         while(cur){
22             stk.push(cur);
23             cur = cur->next;
24         }
25         for(int i = 0; i < n; i++) stk.pop();
26         ListNode* prev = stk.top();
27         prev->next = prev->next->next;
28         ListNode* ans = dummy->next;
29         delete dummy;
30         return ans;
31     }
32 };

```

删除倒数第 n 个结点 用栈这种数据结构解决 先循环全部入栈 然后循环 n 次把倒数第 n 个元素的前一个元素取出 令 $pre \rightarrow next = pre \rightarrow next \rightarrow next$ 即可 此题使用一个哨兵结点 $dummy$ 为了去掉删除节点为头节点这种特殊情况 使用 $dummy$ 之后可以避免对头节点进行特殊处理

238. 除了自身以外数组的乘积

已解答

中等 相关标签 相关企业 提示 Aa

给你一个整数数组 $nums$ ，返回 数组 $answer$ ，其中 $answer[i]$ 等于 $nums$ 中除了 $nums[i]$ 之外其余各元素的乘积。

题目数据 保证 数组 $nums$ 之中任意元素的全部前缀元素和后缀的乘积都在 32 位 整数范围内。

请 不要使用除法，且在 $O(n)$ 时间复杂度内完成此题。

示例 1:

输入: $nums = [1,2,3,4]$
输出: $[24,12,8,6]$

示例 2:

输入: $nums = [-1,1,0,-3,3]$
输出: $[0,0,9,0,0]$

```

1  class Solution {
2  public:
3      vector<int> productExceptSelf(vector<int>& nums) {
4          int len = nums.size();
5          if(len == 0) return {};
6          vector<int> ans(len);
7          ans[0] = 1;
8          for(int i = 1; i < len; i++){
9              ans[i] = ans[i-1] * nums[i-1];
10         }
11         // 最开始右边没有元素
12         int right = 1;
13         for(int i = len-1; i >= 0; i--){
14             ans[i] = ans[i] * right;
15             right *= nums[i];
16         }
17         return ans;
18     }
19 };

```

可以计算出下标 i 处左侧 $left[i]$ 乘积与右侧 $right[i]$

然后 $ans[i] = left[i] * right[i]$

可以优化空间复杂度

直接向 ans 中累乘即可 不必申请 $left$ 与 $right$ 两个数组

正向累乘后反向累乘

303. 区域和检索 - 数组不可变

已解答

简单 相关标签 相关企业 Aa

给定一个整数数组 `nums`，处理以下类型的多个查询：

- 计算索引 `left` 和 `right`（包含 `left` 和 `right`）之间的 `nums` 元素的和，其中 `left <= right`

实现 `NumArray` 类：

- `NumArray(int[] nums)` 使用数组 `nums` 初始化对象
- `int sumRange(int left, int right)` 返回数组 `nums` 中索引 `left` 和 `right` 之间的元素的总和，包含 `left` 和 `right` 两点（也就是 `nums[left] + nums[left + 1] + ... + nums[right]`）

示例 1：

输入：

```
["NumArray", "sumRange", "sumRange", "sumRange"]
```

```

1 class NumArray {
2 public:
3     vector<int> sums;
4     NumArray(vector<int>& nums) {
5         int len = nums.size();
6         sums.resize(len+1);
7         for(int i = 0; i<len; i++){
8             sums[i+1] = sums[i] + nums[i];
9         }
10    }
11
12    int sumRange(int left, int right) {
13        //从[left,right]的数
14        return sums[right+1] - sums[left];
15    }
16 };
17
18 /**
19  * Your NumArray object will be instantiated and called as such:
20  * NumArray* obj = new NumArray(nums);
21  * int param_1 = obj->sumRange(left,right);
22  */

```

$Pre[i] - pre[j-1]$ 就是 $j \sim i$ 之间的和

利用前缀和解决

1413. 逐步求和得到正数的最小值

已解答

简单 相关标签 相关企业 提示 Aa

给你一个整数数组 `nums`。你可以选定任意的正数 `startValue` 作为初始值。

你需要从左到右遍历 `nums` 数组，并将 `startValue` 依次累加上 `nums` 数组中的值。

请在确保累加和始终大于等于 1 的前提下，选出一个最小的正数作为 `startValue`。

示例 1：

输入：nums = [-3,2,-3,4,2]

输出：5

解释：如果你选择 `startValue = 4`，在第三次累加时，和小于 1。

累加和		
startValue = 4	startValue = 5	nums
(4 -3) = 1	(5 -3) = 2	-3
(1 +2) = 3	(2 +2) = 4	2
(3 -3) = 0	(4 -3) = 1	-3
(0 +4) = 4	(1 +4) = 5	4
(4 +2) = 6	(5 +2) = 7	2

```

1 class Solution {
2 public:
3     int minStartValue(vector<int>& nums) {
4         //前缀和pre[i] 前i个数
5         int len = nums.size();
6         if(len == 0) return 0;
7         vector<int> pre(len+1,0);
8         for(int i = 1; i<=len; i++){
9             pre[i] = pre[i-1] + nums[i-1];
10        }
11        int startValue = INT_MAX;
12        for(auto preNum:pre){
13            startValue = min(startValue,preNum);
14        }
15        return 1-startValue;
16    }
17 };

```

前缀和 求出在哪个下标位置上 $pre[i]$ 最小 用 1 减去这个值即可

3702. 按位异或非零的最长子序列

已解答

中等

相关标签

相关企业

提示

Az

给你一个整数数组 `nums`。

返回 `nums` 中 按位异或 (XOR) 计算结果 非零 的最长子序列 的长度。如果不存在这样的子序列，返回 0。

子序列 是一个 非空 数组，可以通过从原数组中删除一些或不删除任何元素（不改变剩余元素的顺序）派生而来。

示例 1:

输入: `nums = [1,2,3]`

输出: 2

解释:

最长子序列之一是 `[2, 3]`。按位异或计算为 `2 XOR 3 = 1`，它是非零的。

示例 2:

输入: `nums = [2,3,4]`

C++

智能模式

```
1 class Solution {
2 public:
3     bool isAllZero(vector<int>& nums){
4         for(auto num:nums){
5             if(num != 0)return false;
6         }
7         return true;
8     }
9     bool flag = false;
10    int longestSubsequence(vector<int>& nums) {
11        //子序列问题 全部元素异或 如果不等于0 nums自身就是个子序列 如果 = 0
12        //1. 元素全都为0
13        //2. 至少一个元素不是0
14        //那就说明 其他所有元素异或之后 再与 这至少一个这个元素异或 == 0
15        //那么其他哪些元素异或 答案并不等于0 所以 else*x = 0 所以长度为n-1
16        int len = nums.size();
17        if(len == 0)return 0;
18        int res = 0;
19        for(auto num:nums){
20            res ^= num;
21        }
22        if(res != 0){
23            return len;
24        }else{
25            flag = isAllZero(nums);
26        }
27        return flag?0:len-1;
28    }
29 };
```

用性质解题 所有元素异或之后 如果

不等于 0:说明数组本身就是个最长子序列 返回 n

如果等于 0:

1:说明有一个元素 x 异或上数组内其他 n-1 个元素后等于 0

那就说明 数组内异或不为 0 的长度为 n-1

2:特殊情况 数组内所有元素都为 0 返回 0 即可

LCR 038. 每日温度

已解答

中等

相关标签

相关企业

Az

请根据每日 气温 列表 `temperatures` ，重新生成一个列表，要求其对应位置的输出为：要想观测到更高的气温，至少需要等待的天数。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

示例 1:

输入: `temperatures = [73,74,75,71,69,72,76,73]`

输出: `[1,1,4,2,1,1,0,0]`

示例 2:

输入: `temperatures = [30,40,50,60]`

输出: `[1,1,1,0]`

示例 3:

输入: `temperatures = [30,60,90]`

输出: `[1,1,0]`

C++

智能模式

```
1 class Solution {
2 public:
3     vector<int> dailyTemperatures(vector<int>& temperatures) {
4         int len = temperatures.size();
5         //结果集
6         vector<int>ans(len);
7         //单调栈 存下标
8         stack<int>sta;
9         //
10        for(int i =0;i<len;i++){
11            //如果栈不是空的 并且当前元素大于栈顶元素
12            while(!sta.empty() && temperatures[i]>temperatures[sta.top()]){
13                //求出栈顶元素下标
14                int preIndex = sta.top();
15                //栈顶元素下标 = 当前元素下标 - 栈顶元素下标
16                ans[preIndex] = i - preIndex;
17                //栈顶元素弹出
18                sta.pop();
19            }
20            sta.push(i);
21        }
22        return ans;
23    }
24 };
```

单调栈解题 元素下标循环入栈 循环判断当前将要入栈元素是否大于栈顶元素

小于栈顶元素 当前元素下标入栈 继续遍历

大于栈顶元素 栈顶元素出栈 用当前元素下标 - 栈顶元素下标得到栈顶元素
`ans[topIndex]`; 继续判断当前元素是否大于新的栈顶元素